团队技术分享

构建详情状态小结技术方案

适用接口：getBuildRecordByExecuteCount / ModelRecord

覆盖范围：运行态 buildRunningInfo，终态 buildEndInfo

资料来源：构建详情状态小结方案设计、终态/运行态前端对接文档

在同一条构建详情接口中，为运行中与已结束构建提供一张通俗、稳定、可跳转的状态小结卡片。

01 BACKGROUND

为什么要做这张小结卡片

过去详情页只给出“失败、已取消、运行中”等状态词，用户还要翻编排图和日志才能判断构建卡在哪里、是谁取消的、到底失败在哪个插件。

运行中

现在卡在哪

排队时展示队列位置、占用构建、前方排队；执行时展示等待中的 Job 和待人工处理项。

已结束

为什么结束

明确取消、失败、超时、成功的大类和子类型，失败原因落到具体 Stage / Job / 插件。

前端友好

拿字段直接渲染

后端给出翻译后的标签、原因、位置和跳转参数，前端不再从状态与错误码里反推成因。

本次方案的核心目标

目标	解释	用户收益
一张卡片	同一时刻只返回 buildRunningInfo 或 buildEndInfo	页面不出现两张互相打架的状态卡
口径一致	构建级大类始终与页面左上角最终状态同类	避免“失败状态 + 取消卡片”的困惑
能跳转	位置级带 stageId、containerId、taskId、关联构建信息	用户从小结直接定位到具体组件或关联流水线
易扩展	前端依赖 category 与 desc，不硬编码每个子类型	后续新增类型时改动面小

02 PRINCIPLE

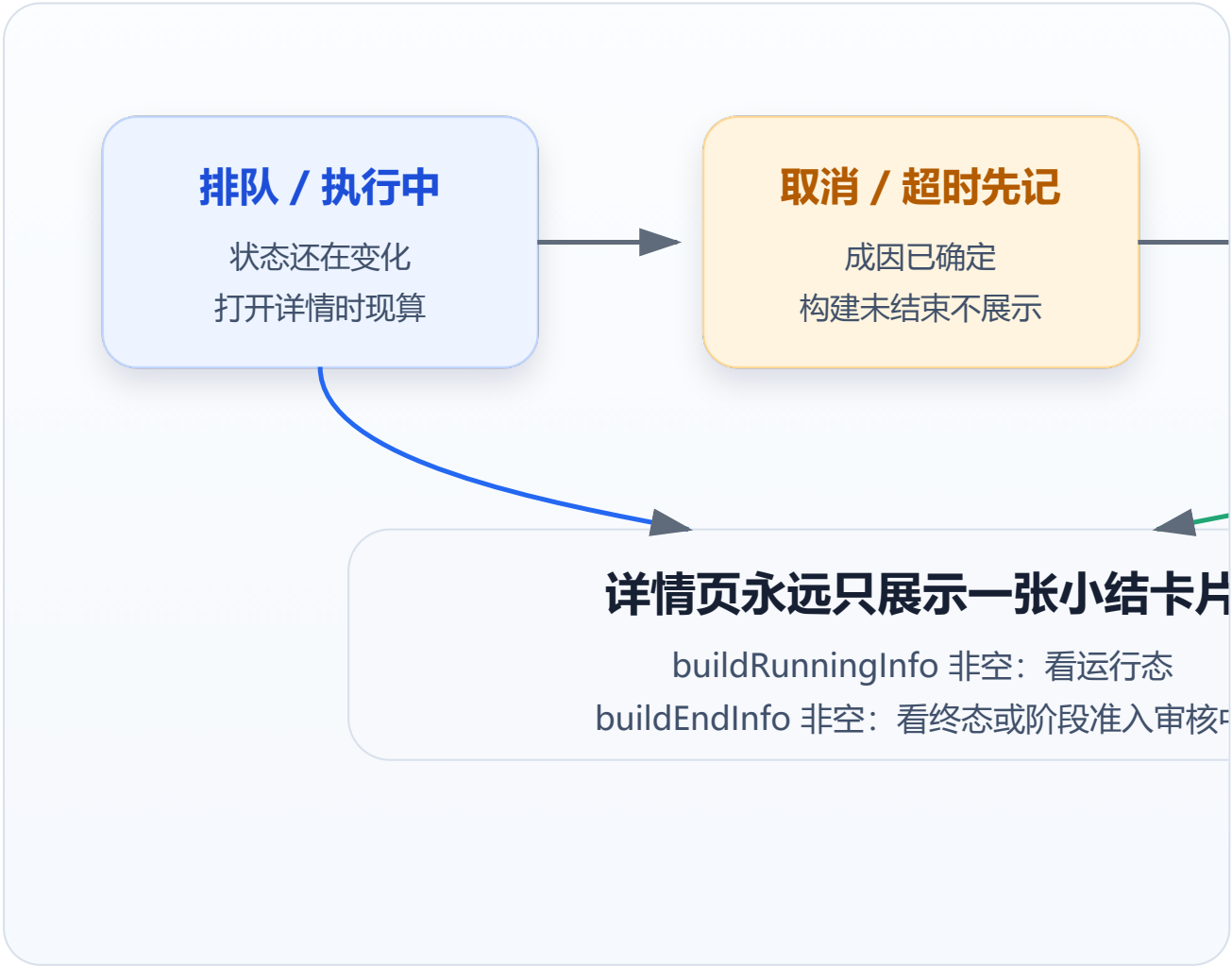
设计原则：运行态实时算，终态稳定记

buildRunningInfo：实时快照

- 只在排队中或执行中返回。
- 不落库，打开详情时现算。
- 排队位置、等待时长、前方队列每秒都可能变化。
- 异常时返回空，不影响详情页打开。

buildEndInfo：终态结论

- 取消、失败、排队超时、阶段准入等场景返回。
- 成因确定后写入 modelVar，普通成功读取时合成。
- 构建仍在跑时，即使提前写过终态成因也不展示。
- 读取时会按最终状态校正大类。



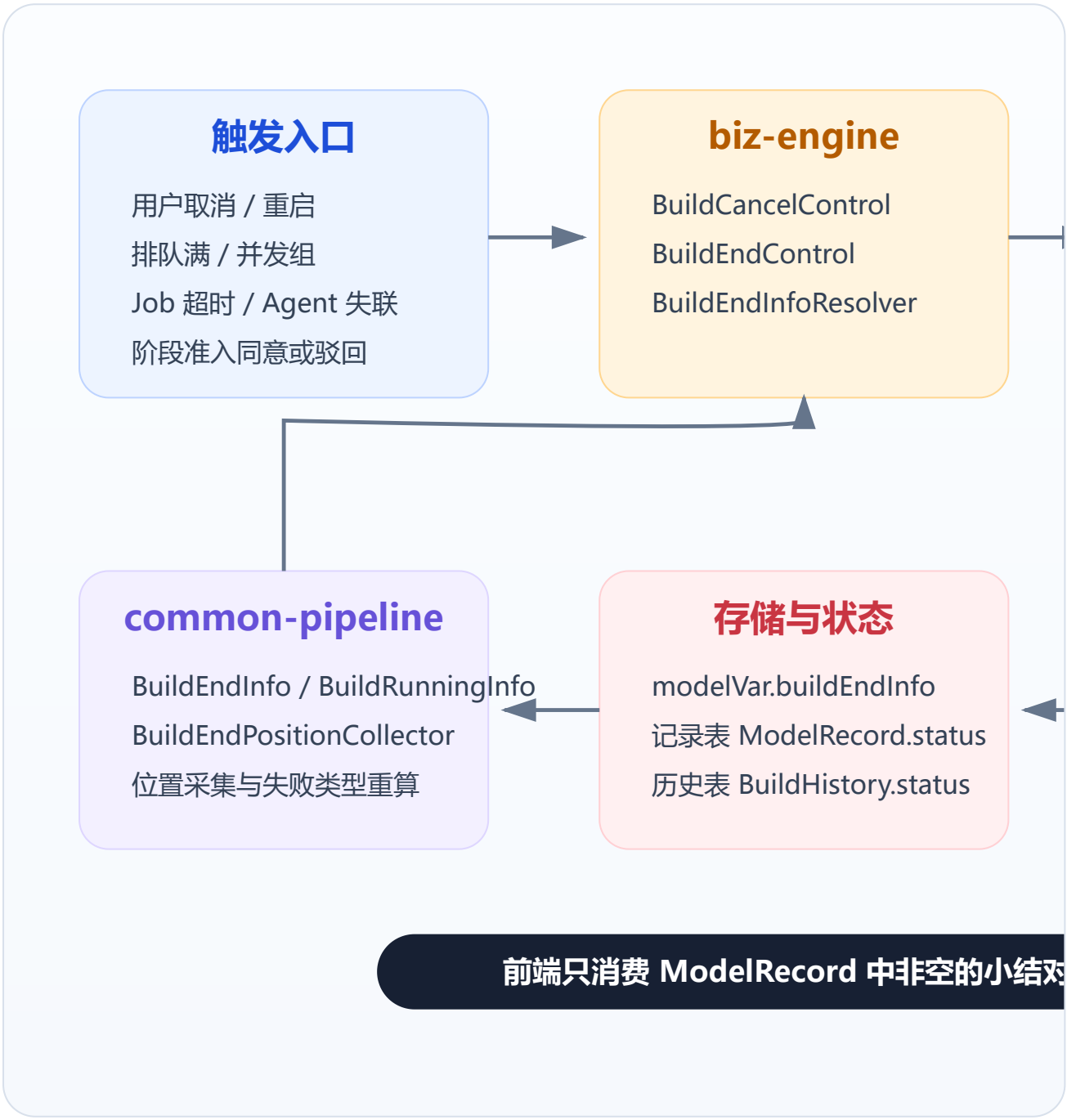
一句话总结：运行态解决“现在在哪里等”，终态解决“最终为什么结束”。两者字段语义不同，也不共用位置结构。

构建详情状态小结技术方案

03 ARCHITECTURE

模块分工与数据流

方案复用既有详情接口，不新增接口。写入侧由引擎和记录服务捕获成因，读取侧由 Facade 组装运行态或终态卡片。

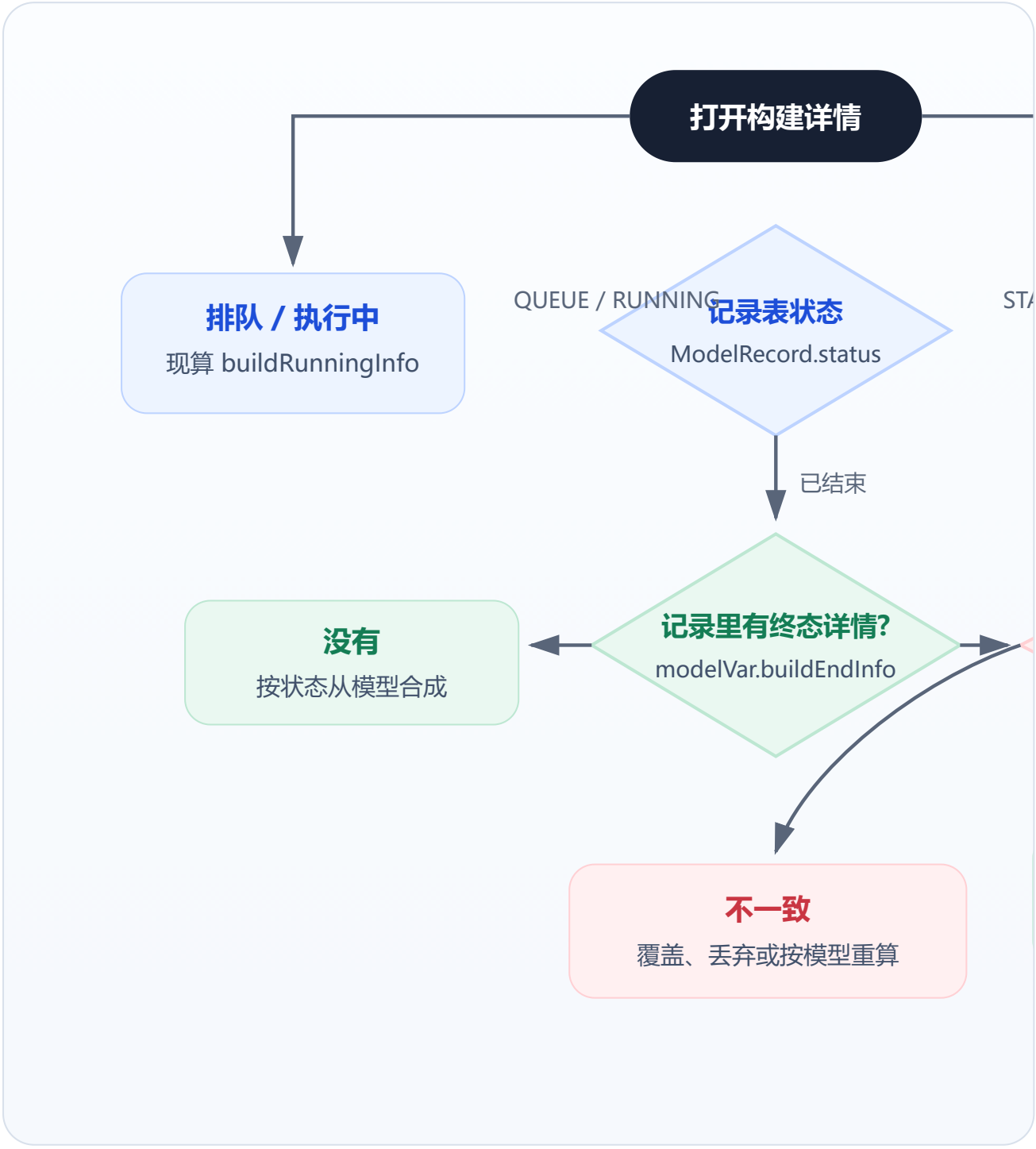


构建详情状态小结技术方案

04 DECISION FLOW

打开详情时如何决定返回哪张卡片

● 运行态现算 ● 阶段准入挂起 ● 终态返回 ● 对齐或重算



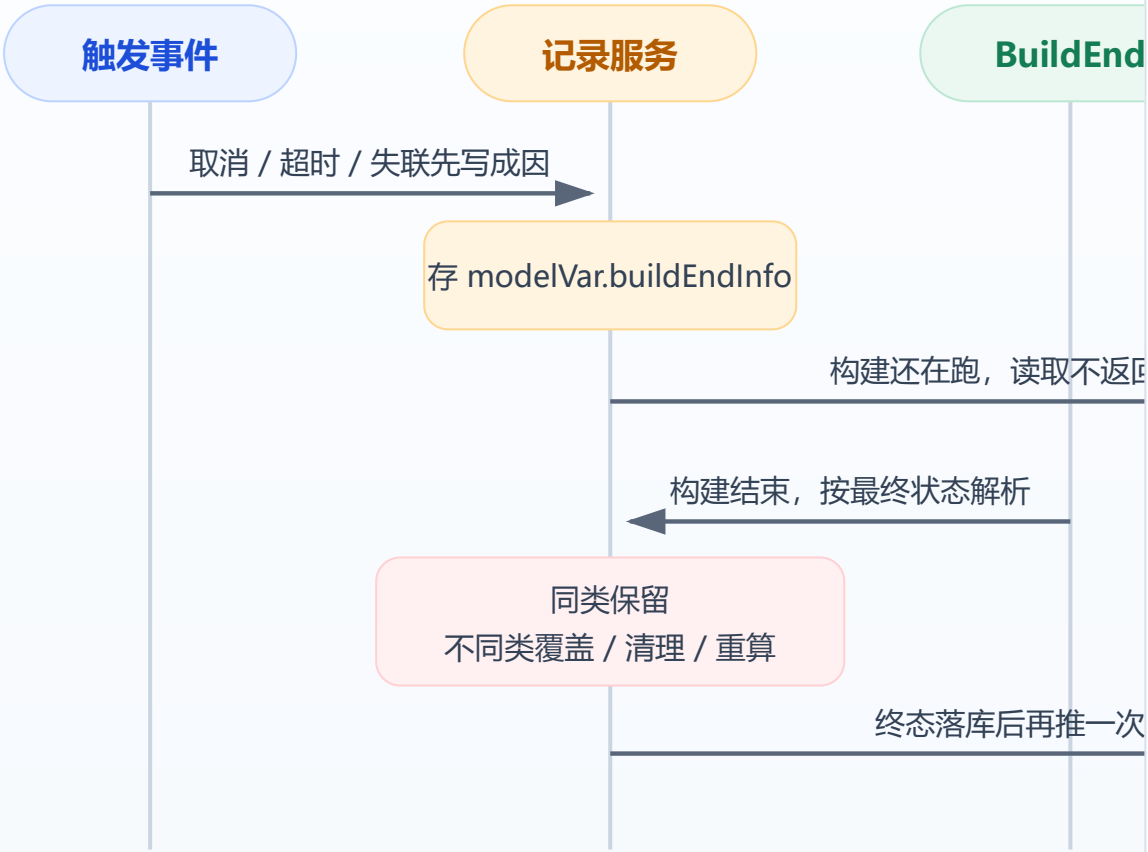
页面状态	返回字段	展示重点
排队中	`buildRunningInfo`	第几位、占用中、前方排队、并发组
执行中	`buildRunningInfo`	当前阶段、等待 Job、待人工处理项

阶段准入审核中	`buildEndInfo`	审核组、审核人、已等待时长
成功 / 失败 / 取消 / 超时	`buildEndInfo`	结束成因、影响位置、跳转信息

05 WRITE PATH

终态信息如何写入与校正

写入策略不是“构建结束时统一扫一遍”这么简单。部分成因只有事件发生当下才准确，例如取消人、父流水线、构建机失联和 Job 超时位置。



先写的场景

- 用户取消、强制终止、重启前取消。
- 系统取消、排队满、并发组挤掉、服务关闭。
- 父流水线取消子构建，带父流水线信息。

结束时统一对齐

- 失败、阶段准入驳回、排队超时写最终结论。
- 普通成功不写库，读取时合成成功卡片。
- Job/步骤超时若最终是失败，构建级按失败展示，位置保留超时原

- Job 超时、构建机 Agent 失联、执行前暂停被终止。

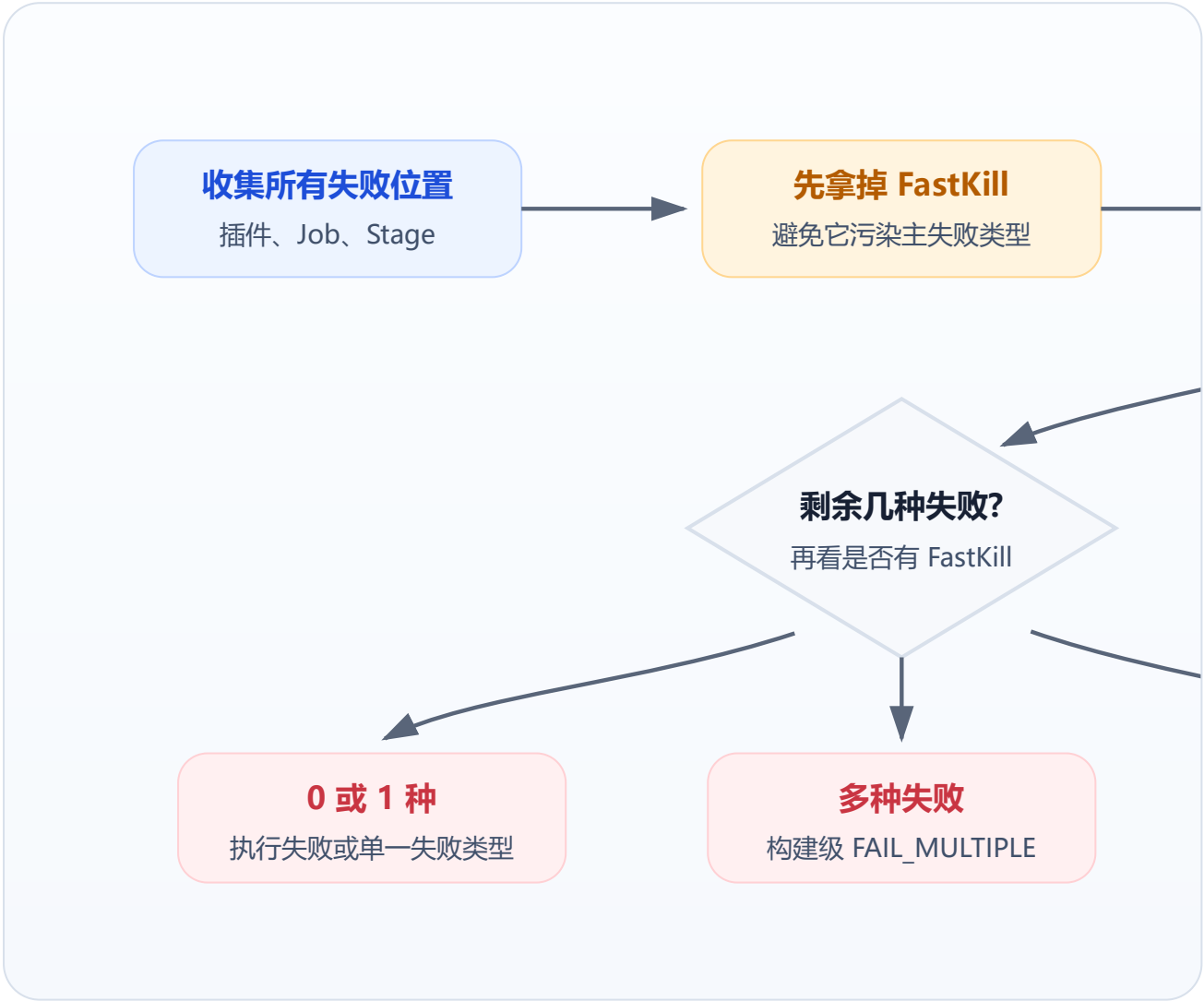
因。

- 重试开始时清掉本轮已写的旧终态，避免串轮次。

06 FAILURE STRATEGY

失败分类：构建级看结论，位置级讲原因

失败场景最容易误判。方案把“构建最终属于哪一类失败”和“每个位置为什么失败”拆开处理，避免用一个原因覆盖所有失败点。



位置上看到的情况	构建级展示	原因展示
一个插件执行失败	执行失败	位置上展示错误码或位置原因
步骤超时 / Job 超时导致最终失败	执行失败	位置保留“步骤超时 / Job 超时”和分钟数
插件失败 + 失败即停连带终止	多类失败	失败行展示插件错误，被终止行展示 FastKill 原因
质量红线 + 人工审核驳回	多类失败	每条位置展示自己的红线指标或驳回意见

模型里收集不到具体位置	执行失败兜底	保证详情页仍能展示失败卡片

> 失败类构建级 reason 恒为空。页面不要因为 reason 为空隐藏卡片，而应遍历 positions 展示各位置原因。

07 RUNNING STATE

运行态：告诉用户“现在在等什么”

排队中 buildRunningInfo

主信息：第几位、已等待多久、何时入队

并发信息：并发组、最大并发数、谁正在占用

队列列表：占用中最多 10 条，前方排队最多 10 条

跳转：关联构建必须用 brief.projectId

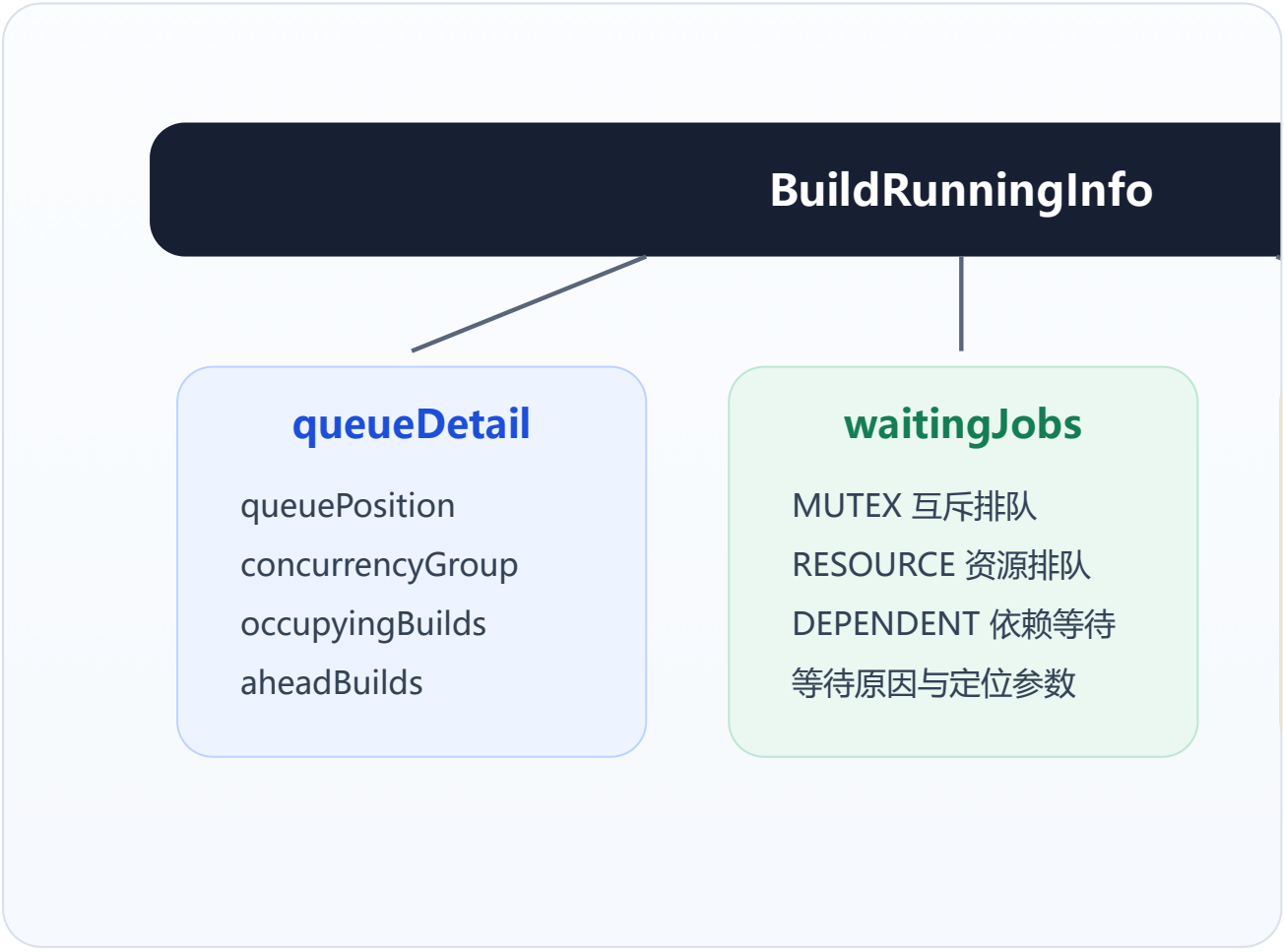
执行中 buildRunningInfo

主信息：当前阶段、已运行多久、开始时间

等待 Job：互斥排队、资源排队、依赖等待

待处理：执行前暂停、人工审核、质量红线、阶段审核

跳转：用 stageId / containerId / taskId 定位



两个关键细节

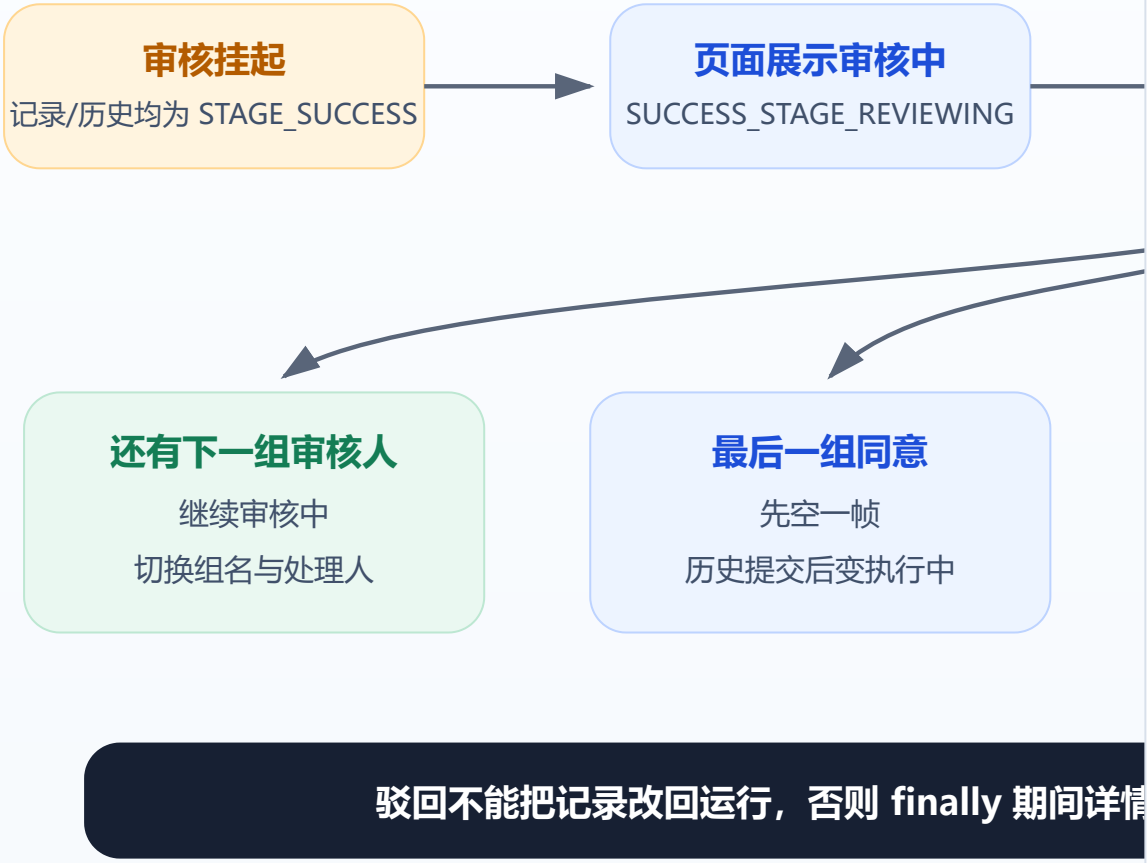
时长由前端自增。 后端返回 waitingTime / runningTime 和 currentTimestamp，前端每秒加即可，不需要为了秒级刷新高频轮询接口。

准备环境有 15 秒缓冲。 正常拉起构建机的短暂准备不算资源排队，避免每个构建都被误标成卡住。

08 STAGE REVIEW

阶段准入：最容易看错的状态切换

阶段准入会让记录表和历史表短暂不一致。详情页以记录表状态作为终态判断，以历史表辅助运行态计算。



已知空窗是推送时序问题，不是状态写错。前端按最后一次推送渲染即可。

09 FRONTEND

前端渲染模型

buildEndInfo 终态卡

样式： endCategory 决定 CANCEL / FAIL / TIMEOUT / SUCCESS

标签： endTypeDesc 直接展示，不硬编码中文

原因： 失败类看 positions.reason，非失败类可看构建级 reason

位置： positions 最多 50 条，支持定位组件、日志、父子流水线

buildRunningInfo 运行卡

样式： runningCategory 决定 QUEUE / RUNNING

标题： runningTypeDesc，排队态不要拼并发组名

时长： 结合 currentTimestamp 自增

列表： 展示计数字段，不用 list.length 代表真实总数

字段与页面区域对应

页面区域	终态字段	运行态字段	注意事项
卡片标题/配色	endCategory + 页面状态标签	runningCategory + runningTypeDesc	不要用子类型硬推大类
成因标签	endTypeDesc	waitingJobs.waitTypeDesc / pendingItems.itemTypeDesc	字段已翻译
原因详情	reason 或 positions.reason	waitReason / reviewDesc	errorMsg 只适合 tooltip 或展开态
跳转定位	stageId / containerId / taskId / parentPipelineInfo / subPipelineInfo	stageId / containerId / taskId / dependOnJobs / brief.projectId	跨流水线必须用对象自己的 projectId

数量展示	positionCount	aheadCount / occupyingCount / waitingJobCount / pendingItemCount	列表可能被截断

推荐渲染分支

> **先判断 buildEndInfo，再判断 buildRunningInfo。** 阶段准入审核中虽然构建未真正结束，但使用 buildEndInfo 表达。正常情况下两者互斥，分支顺序不会影响结果，优先终态可以让审核中语义更清晰。

10 ROLLOUT

上线验证与风险控制

发布范围

- common-pipeline：数据结构、位置采集与失败类型重算。
- biz-base：记录读写、运行态现算、阶段准入处理。
- biz-engine：取消、超时、心跳、构建结束写入。
- biz-process：详情接口组装。
- support-files/i18n：新增 buildEndType、buildRunningType 与 reasonCode 文案。

已知窗口

- 阶段准入最后一组同意后，可能空一帧。
- 驳回后到 finally 结束前，详情保持已取消 / 系统取消。
- 驳回且以阶段成功收尾时，可能先闪一张运行中卡片，再推送阶段准入驳回。

建议 QA 场景

验证主题	关键期望
取消链路	用户取消、强制终止、重启前取消、父流水线取消均能展示正确成因与操作人/父构建信息。
失败链路	步骤超时或 Job 超时最终失败时，构建级为执行失败，位置级保留超时类型和分钟数。
FastKill	插件失败导致同阶段其他 Job 被终止时，构建级为多类失败，被终止行显示 FastKill 原因。
阶段准入	审核中、最后同意、驳回、finally 后阶段成功驳回的卡片切换符合预期。
运行态	排队位置、占用构建、前方排队、等待 Job、待人工处理项均可展示并定位。
重试	运行中重试后，上一轮失联或 Job 超时卡片不再污染本轮。

上线结论：方案可上线。需要关注 biz-base / biz-engine 在 CI 中完成 jOOQ 依赖编译验证。

11 SUMMARY

分享总结

1
一条接口，两类卡片

继续复用 getBuildRecordByExecuteCount，不新增接口。运行态与终态互斥返回，前端按非空对象渲染。

2
状态大类与页面一致

构建最终是失败就展示失败卡，最终是取消就展示取消卡。超时、失联、FastKill 等具体成因放到原因和位置上。

3
复杂场景前后端分工清晰

后端负责归因、翻译、位置采集与跳转参数；前端负责样式、布局、判空和跨项目跳转。

4
可验证、可扩展

QA 可以按取消、失败、阶段准入、运行排队和重试几个主线验收；后续新增子类型不影响现有大类渲染。

排队中

第 3 位 · 已等待 12 秒

占用中 1 · 前方排队 2

并发组 release-train

执行失败

多类失败

质量红线 / FastKill / 子流水线

点击位置直达日志